UNIT I
Principles of Object-Oriented Programming & Control Structures

SECTION A — DEBUGGING QUESTIONS

Question 1 — Operator precedence and unsigned countdown

Precedence (Line 7): == has higher precedence than bitwise &, so num & 1 == 1 parses as num & (1 == 1), i.e. num & 1. This happens to test the low bit after bool conversion, but it is unclear and omits the explicit comparison. Write ((num & 1u) == 1u) and combine conditions with &&.

Unsigned loop (Line 14): unsigned char cannot represent −1. Decrementing zero wraps to its maximum value, so i >= 0 remains true. Use a signed counter.

```
#include <iostream>
int main() {
   unsigned char num = 15;
   if (((num & 1u) == 1u) && num > 10u)
      std::cout << "Valid Odd Number: " << static_cast<int>(num) << '\n';
   else
      std::cout << "Condition Failed for: " << static_cast<int>(num) << '\n';

   for (int i = 3; i >= 0; --i)
      std::cout << i << (i == 0 ? '\n' : ' ');
}
```

Dry-run check	Result
15 & 1 == 1; 15 > 10	Both true
Signed i: 3, 2, 1, 0	Stops after decrement to −1

Output:

```
Valid Odd Number: 15
3 2 1 0
```

Question 2 — Scope resolution and variable shadowing

Line 11 is a syntax error: the global scope operator is ::, not :. The expression ::taxRate always names the global variable (0.18); it cannot access an enclosing function-local variable. In the nested block, an inner declaration named taxRate hides the outer local name. Rename variables to make intent explicit.

```
#include <iostream>
#include <iomanip>
double taxRate = 0.18;
int main() {
   const double itemPrice = 1000.0;
   const double subsidizedRate = 0.05;
   std::cout << std::fixed << std::setprecision(2);
   std::cout << "Local Rate: " << subsidizedRate << '\n';
   const double totalWithGlobalTax = itemPrice + itemPrice * ::taxRate;
   {
      const int corporateTaxPercent = 12;
      const double totalWithLocalTax = itemPrice + itemPrice * subsidizedRate;
      std::cout << "Inner total: " << totalWithLocalTax << '\n';
      std::cout << "Corporate rate: " << corporateTaxPercent << "%\n";
   }
   std::cout << "Final Global Calculation: " << totalWithGlobalTax << '\n';
}
Local Rate: 0.05
Inner total: 1050.00
Corporate rate: 12%
Final Global Calculation: 1180.00
```

Question 3 — Switch initialization and illegal case type

Without braces, declarations in a switch case can be in scope across later labels. A jump to another case may bypass initialization, which is illegal. Put each case body in braces. switch expressions must be integral or enum types after promotion; case labels must be compatible constant expressions, so case 4.5 is invalid. Missing break after case 2 causes fall-through to case 3 and adds 100.

```
#include <iostream>
int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;
    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }
        case 3: {
            tokenValue += 100.0f;
            break;
        }
        case 4: {
            std::cout << "Integer token category 4 detected\n";
            break;
        }
        default:
            std::cout << "Unknown token\n";
    }
    std::cout << "Calculated Value: " << tokenValue << '\n';
}
```

Output:

```
Calculated Value: 100
```

Question 4 — Reference initialization and const binding

Invalid line	Rule violated	Valid approach
int &refA;	Reference must be initialized at declaration.	int& refA = a;
int &refLiteral = 100;	Non-const lvalue reference cannot bind to an rvalue.	const int& refLiteral = 100;
int &refInt = pi;	int& cannot bind to a double object or converted temporary.	double& refPi = pi; or const int& to an explicit converted temporary.
constRef = constRef + 10;	Cannot modify through const reference.	Do not assign through constRef.

A const lvalue reference can bind to a temporary. When directly initialized from a temporary (including a converted temporary), the temporary's lifetime is extended to the lifetime of the local reference. The referred value remains read-only through that reference.

```
#include <iostream>
int main() {
    int a = 25;
    int& refA = a;
    const int& refLiteral = 100;
    double pi = 3.14159;
    double& refPi = pi;
    const int& refInt = static_cast<int>(pi);
    const int& constRef = 50;
    // constRef = constRef + 10; // compile-time error
    refA = 30;
    refPi = 3.14;
    std::cout << a << " " << refA << '\n';
    std::cout << refLiteral << " " << refInt << " " << constRef << '\n';
    std::cout << pi << '\n';
}
```

Output:

```
30 30
100 3 50
3.14
```

Question 5 — Dynamic array and stray loop semicolon

The semicolon after the for header makes the loop body empty. The following block is separate, and i declared in the for initializer is out of scope there, causing a compile error. Also, memory allocated with new[] must be released with delete[]; using scalar delete on an array is undefined behavior.

```
#include <iostream>
int main() {
    const int size = 5;
    int* readings = new int[size];
    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }
    int total = 0;
    for (int i = 0; i < size; ++i)
        total += readings[i];
    std::cout << "Total: " << total << '\n';
    delete[] readings;
}
```

i	Reading	Running total
0	10	10
1	20	30
2	30	60
3	40	100
4	50	150

Output:

```
Total: 150
```

SECTION B — STRUCTURED PSEUDOCODE QUESTIONS

Question 1 — Procedural-to-OOP water consumer billing

Design: keep consumer and meter data private; validate readings; calculate consumption; apply progressive unit tiers. The 10% surcharge applies to the entire base bill only when consumption exceeds 300.

```
CLASS WaterConsumer
 PRIVATE:
  consumerID : STRING
  previousReading, currentReading : REAL
  unitsConsumed, totalBill : REAL
  valid : BOOLEAN
 PUBLIC:
  METHOD Initialize(id : STRING, prev : REAL, curr : REAL)
   IF prev < 0 OR curr < 0 OR curr < prev THEN
    valid <- FALSE
    PRINT "ERROR: Invalid meter readings"
    RETURN
   ENDIF
   consumerID <- id
   previousReading <- prev
   currentReading <- curr
   unitsConsumed <- 0
   totalBill <- 0
   valid <- TRUE
  ENDMETHOD

  METHOD CalculateConsumption()
   IF valid = FALSE THEN RETURN
   unitsConsumed <- currentReading - previousReading
  ENDMETHOD

  METHOD ComputeBill()
   IF valid = FALSE THEN RETURN
   IF unitsConsumed <= 100 THEN
    totalBill <- unitsConsumed * 1.50
   ELSE IF unitsConsumed <= 300 THEN
    totalBill <- 100 * 1.50 + (unitsConsumed - 100) * 2.50
   ELSE
    totalBill <- 100 * 1.50 + 200 * 2.50
    totalBill <- totalBill + (unitsConsumed - 300) * 4.00
    totalBill <- totalBill * 1.10
   ENDIF
  ENDMETHOD

  METHOD DisplayBill()
   IF valid THEN
    PRINT consumerID, unitsConsumed, totalBill
   ELSE
    PRINT "Invoice unavailable: invalid readings"
   ENDIF
  ENDMETHOD
ENDCLASS

BEGIN MAIN
 customer <- WaterConsumer()
 customer.Initialize("WC-001", 1450.0, 1820.0)
```

```
customer.CalculateConsumption()
customer.ComputeBill()
customer.DisplayBill()
END MAIN
```

Tier / step	Calculation	Amount
Consumption	1820 − 1450	370 units
First 100	100 × 1.50	$150
Next 200	200 × 2.50	$500
Above 300	70 × 4.00	$280
Base bill	150 + 500 + 280	$930
Surcharge	10% × 930	$93
Total	930 + 93	$1,023

Question 2 — Multi-tier tax and precision cast engine

Design: clamp deductions to their statutory caps, clamp taxable income to zero, calculate tax marginally by bracket, add 4% cess, and round to the nearest integer using INTEGER(tax + 0.5).

```
CLASS TaxCalculationEngine
 PUBLIC:
  METHOD Calculate(gross : REAL, d80C : REAL, health : REAL) RETURNS INTEGER
    eligible80C <- MIN(MAX(d80C, 0), 150000)
    eligibleHealth <- MIN(MAX(health, 0), 25000)
    taxable <- MAX(0, gross - eligible80C - eligibleHealth)
    rawTax <- 0
    IF taxable > 250000 THEN
      rawTax <- rawTax + MIN(taxable - 250000, 250000) * 0.05
    ENDIF
    IF taxable > 500000 THEN
      rawTax <- rawTax + MIN(taxable - 500000, 500000) * 0.20
    ENDIF
    IF taxable > 1000000 THEN
      rawTax <- rawTax + (taxable - 1000000) * 0.30
    ENDIF
    cess <- rawTax * 0.04
    RETURN INTEGER(rawTax + cess + 0.5)
  ENDMETHOD
ENDCLASS
BEGIN MAIN
 engine <- TaxCalculationEngine()
 PRINT engine.Calculate(1200000, 180000, 30000)
END MAIN
```

Item	Calculation	Value
80C eligible	MIN(180000,150000)	150,000
Health eligible	MIN(30000,25000)	25,000
Taxable income	1,200,000−150,000−25,000	1,025,000
0% band	First 250,000	0
5% band	250,000 × 5%	12,500
20% band	500,000 × 20%	100,000
30% band	25,000 × 30%	7,500

Raw tax	12,500+100,000+7,500	120,000
Cess	120,000 × 4%	4,800
Rounded total	120,000+4,800	124,800

Output: 124800.

Question 3 — Bitwise mask validation and UAV telemetry

Required mask is 0x0F (bits 0–3). Inhibitor mask is 0xB0 (bits 4, 5, 7). Radio bit 0x40 is non-blocking. AND tests flags, OR combines masks, XOR identifies missing required bits, and NOT can be restricted to one byte with 0xFF.

```
CONSTANT GYRO=0x01, ACCEL=0x02, BAROMETER=0x04, GPS=0x08
CONSTANT LOW_BATTERY=0x10, OVER_TEMP=0x20, RADIO=0x40, ACTUATOR=0x80
CONSTANT REQUIRED_MASK=GYRO OR ACCEL OR BAROMETER OR GPS
CONSTANT INHIBITOR_MASK=LOW_BATTERY OR OVER_TEMP OR ACTUATOR
CONSTANT BYTE_MASK=0xFF

METHOD CheckFlightReadiness(statusRegister : BYTE)
 missingMask <- REQUIRED_MASK XOR (statusRegister AND REQUIRED_MASK)
 active <- statusRegister AND INHIBITOR_MASK
 IF (active AND LOW_BATTERY) != 0 THEN PRINT "Low battery (bit 4)"
 IF (active AND OVER_TEMP) != 0 THEN PRINT "Over-temperature (bit 5)"
 IF (active AND ACTUATOR) != 0 THEN PRINT "Fatal actuator failure (bit 7)"
 IF missingMask != 0 THEN PRINT "Missing sensor mask: ", HEX(missingMask)
 clearedBits <- (NOT statusRegister) AND BYTE_MASK
 IF missingMask = 0 AND active = 0 THEN
   PRINT "FLIGHT READY"
 ELSE
   PRINT "FLIGHT NOT READY"
 ENDIF
ENDMETHOD
```

Test byte	Required / missing	Inhibitors	Result
0x4F	0x4F AND 0x0F = 0x0F; missing = 0x00	0x4F AND 0xB0 = 0x00	Ready; radio bit 6 set but non-blocking
0x3B	0x3B AND 0x0F = 0x0B; missing = 0x04	0x3B AND 0xB0 = 0x30	Not ready: bit 2 missing, low battery and over-temperature active

For 0x3B, the XOR difference 0x04 identifies the missing barometric altimeter (bit 2). The active inhibitor mask 0x30 identifies bits 4 and 5.

Question 4 — Optimized divisor and classification engine

Design: validate N > 0; handle N = 1 separately; initialize the proper-divisor sum to 1; search only through floor(sqrt(N)); add divisor pairs without double-counting a square root.

```
BEGIN DivisorAndPrimeEngine
 REPEAT
   INPUT N
   IF N <= 0 THEN PRINT "Enter a positive integer"
 UNTIL N > 0
 IF N = 1 THEN
   sumOfDivisors <- 0
   PRINT "Deficient Number"
   STOP
 ENDIF
 sumOfDivisors <- 1
 d <- 2
 limit <- FLOOR(SQRT(N))
 WHILE d <= limit DO
   IF N MOD d != 0 THEN
     d <- d + 1
     CONTINUE
   ENDIF
   paired <- N DIV d
   sumOfDivisors <- sumOfDivisors + d
   IF paired != d THEN
     sumOfDivisors <- sumOfDivisors + paired
   ENDIF
   d <- d + 1
 ENDWHILE
 IF sumOfDivisors = N THEN PRINT "Perfect Number"
 ELSE IF sumOfDivisors > N THEN PRINT "Abundant Number"
 ELSE PRINT "Deficient Number"
 ENDIF
END
```

N=28: d	Divides? / pair	Running sum
Start	1 included	1
2	yes; pair 14	17
3	no; CONTINUE	17
4	yes; pair 7	28
5	no; CONTINUE	28

28: proper divisors 1, 2, 4, 7, 14 sum to 28 → Perfect Number.

N=12: d	Divides? / pair	Running sum
Start	1 included	1
2	yes; pair 6	9
3	yes; pair 4	16

12: proper divisors 1, 2, 3, 4, 6 sum to 16 → Abundant Number. Time complexity O(√N); auxiliary space O(1).

Question 5 — ATM state-machine controller

Design: authenticate against PIN 4821 with at most three tries. Only after authentication, enter a sentinel-controlled menu loop. Deposits must be positive; withdrawals must be positive, affordable, and divisible by 100.

```
BEGIN ATM_SessionController
 CONSTANT CORRECT_PIN <- 4821
 balance <- 1000.00  // Assumed opening balance for worked trace
 attempts <- 0
 authenticated <- FALSE
 WHILE attempts < 3 AND authenticated = FALSE DO
  INPUT enteredPIN
  IF enteredPIN = CORRECT_PIN THEN
   authenticated <- TRUE
  ELSE
   attempts <- attempts + 1
   PRINT "Incorrect PIN"
  ENDIF
 ENDWHILE
 IF authenticated = FALSE THEN
  PRINT "Card locked"
  STOP
 ENDIF

 REPEAT
  PRINT "1 Balance 2 Deposit 3 Withdraw 4 Exit"
  INPUT choice
  SWITCH choice
   CASE 1:
    PRINT balance
    BREAK
   CASE 2:
    INPUT amount
    IF amount > 0 THEN
     balance <- balance + amount
     PRINT "Deposit successful"
    ELSE PRINT "Invalid deposit"
    ENDIF
    BREAK
   CASE 3:
    INPUT amount
    IF amount <= 0 THEN PRINT "Invalid amount"
    ELSE IF amount > balance THEN PRINT "Insufficient funds"
    ELSE IF amount MOD 100 != 0 THEN PRINT "Must be a multiple of 100"
    ELSE
     balance <- balance - amount
     PRINT "Withdrawal successful"
    ENDIF
    BREAK
   CASE 4:
    PRINT "Session ended"
    BREAK
   DEFAULT:
    PRINT "Invalid menu option"
  ENDSWITCH
```

```
 UNTIL choice = 4
END
```

Step	Input / action	Outcome	Balance
1	PIN 1111	Wrong PIN; attempt 1	$1,000
2	PIN 4821	Authenticated	$1,000
3	Withdraw $250	Rejected: not a multiple of 100	$1,000
4	Withdraw $300	Successful	$700
5	Exit	Session ended	$700

The opening balance is assumed to be $1,000 because none is supplied in the question bank.